Robot speed & steering Telemetry Analyzer

Owen Rogg, Gracy Sylvestre.

Course: ES115 – Engineering Computer Applications

Module 2: Physics & Chemistry Applications

Date

Gracy's Github

Owen's Github

UNIVERSITY OF HARTFORD

Problem Overview / System Description

- What system are you modeling?

Robot Speed & Steering Telemetry Analyzer system

- Why is this system important in engineering?

This system proves its importance in engineering due to its ability to process and interpret telemetry data whether its synthetic or produced by a real robotic vehicle.

- Real-world application (physics or chemistry context)
- Examples:
  - Rotational Motion (steering angle)
  - Vehicle kinematics
  - Noise smoothing (moving average)

Engineering Objective

- What are you trying to analyze or predict?

The objective of this project is to analyze speed and steering telemetry data

- What question does your model answer?
- Examples:
  - How does the robots motion change over time?
  - How often and at what angle is it turning?
  - When does the robot stop performing?
  - How do the two runs compare?

Mathematical Model

- Present the governing equation(s)

```
velocityRun1 = rand(size(t)) * 10
velocityRun2 = rand(size(t)) * 10
steeringAngleRun1 = -15 + rand(size(t)) * 30
steeringAngleRun2 = -15 + rand(size(t)) * 30
avgVelocityRun1 = movmean(velocityRun1, Moving_window);
avgVelocityRun2 = movmean(velocityRun2, Moving_window);
avgSteeringAngleRun1 = movmean(steeringAngleRun1, Moving_window);
avgSteeringAngleRun2 = movmean(steeringAngleRun2, Moving_window);
stopEventsRun1 = sum(avgVelocityRun1 < stop);
stopEventsRun2 = sum(avgVelocityRun2 < stop);
turnEventsRun1 = sum(abs(avgSteeringAngleRun1 > turn));
turnEventsRun2 = sum(abs(avgSteeringAngleRun2 > turn));
spikesRun1 = sum(abs(diff(avgSteeringAngleRun1)) > spikeThreshold);
spikesRun2 = sum(abs(diff(avgSteeringAngleRun2)) > spikeThreshold);
smoothness1 = var(velocityRun1) / var(avgVelocityRun1);
smoothness2 = var(velocityRun2) / var(avgVelocityRun2);
DQ1 = 100 * (0.4 * smoothScore1 + 0.3 * steeringScore1 + 0.3 * stopScore1);
DQ2 = 100 * (0.4 * smoothScore2 + 0.3 * steeringScore2 + 0.3 * stopScore2);
```

- Define all variables and constants

```
t=[1:dt]'
Moving_window = input("Enter moving average window size (e.g., 15): "); % moving window selection
if isempty(Moving_window); Moving_window = 15; end %default value
stop = input("Enter stop threshold (e.g., 0.1): "); % define stop threshold
if isempty(stop); stop = 0.1; end %default value
turn = input("Enter turning threshold in degrees (e.g., 15): "); %define turn threshold
if isempty(turn); turn = 15; end %default value
spikeThreshold = 7;
smoothScore1 = 1 / (1 + var(diff(avgVelocityRun1)));
smoothScore2 = 1 / (1 + var(diff(avgVelocityRun2)));
steeringScore1 = 1 / (1 + spikesRun1);
steeringScore2 = 1 / (1 + spikesRun2);
stopScore1 = 1 / (1 + stopEventsRun1);
stopScore2 = 1 / (1 + stopEventsRun2);
```

- Time representation (vector)
- Generation of velocity and steering signal
- Moving average filter
- Detection of stop events
- Detection of turning events
- Detection of spike events
- Metric for smoothness
- Driving quality score

Algorithm / System Logic

- Explain how the system works step-by-step

The user inputs the dataset size, moving average window size, stop threshold, and turning threshold.

- "if data set > empty -> data set = 75" , "If moving window > empty> moving window = 15", "if stop threshold > empty> stop threshold= 0.1" ,

"if turn threshold > empt y -> turn threshold= 15"

generate and establish variables such as time, velocity and steering angle for both runs

- implement moving average, determine stop events, turn events, average speed, and display smooth signal results

- "if variance(averagevelocity) < variance(velocity) - > then signal smoothing is valid -> else -> smoothing may be incorrect

- Create visualizations of the system such as a velocity vs, time graph, teering vs time graph, scatter graph of steering vs speed, and histogram of speed.

- Using 2-3 metrics like a smoothness score, steering spike score and, stop event score establish a driving quality score and display score results.
- "if driving score > 80 -> "great control" -> else if-> "if driving score > 60 -> "moderate control" -> else if-> "if driving score > 40 -> "rough driving"-> else ->

MATLAB Implementation

- Show key parts of your code (not everything)
- Highlight:
  - Loops (for/while)
  - Conditional statements
  - Variables and parameters

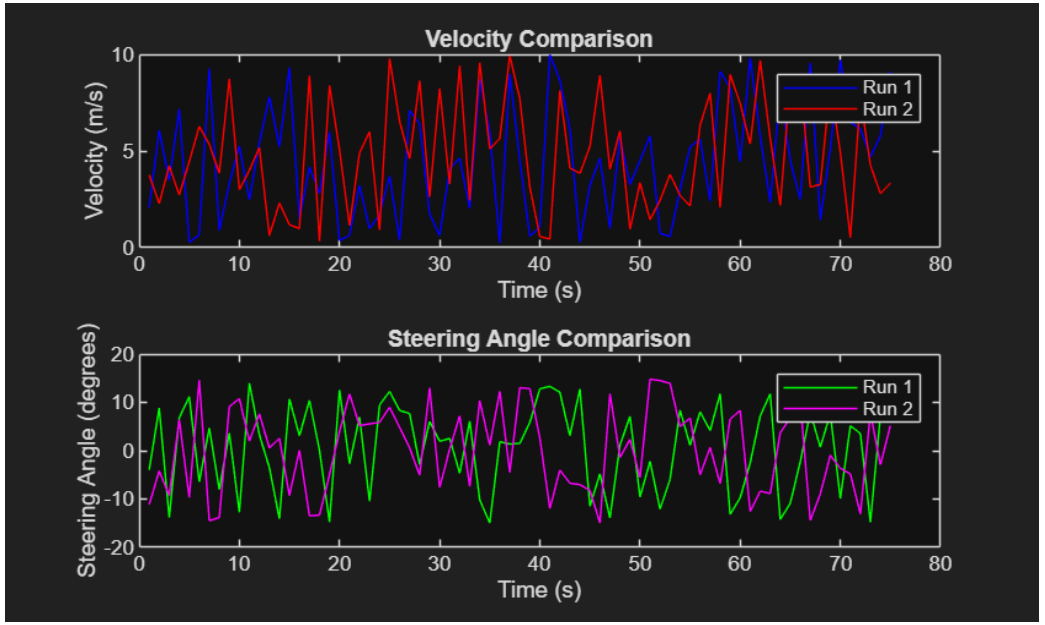
```
dt = input("Enter number of data points (e.g., 75): "); % dataset size
if isempty(dt); dt = 75; end % default value

Moving_window = input("Enter moving average window size (e.g., 15): "); % moving window selection
if isempty(Moving_window); Moving_window = 15; end %default value

stop = input("Enter stop threshold (e.g., 0.1): "); % define stop threshold
if isempty(stop); stop = 0.1; end %default value

turn = input("Enter turning threshold in degrees (e.g., 15): "); %define turn threshold
if isempty(turn); turn = 15; end %default value

%moving average filter
% Calculate the average velocity and steering angle over the moving window
avgVelocityRun1 = movmean(velocityRun1, Moving_window);
avgVelocityRun2 = movmean(velocityRun2, Moving_window);


avgSteeringAngleRun1 = movmean(steeringAngleRun1, Moving_window);
avgSteeringAngleRun2 = movmean(steeringAngleRun2, Moving_window);
```

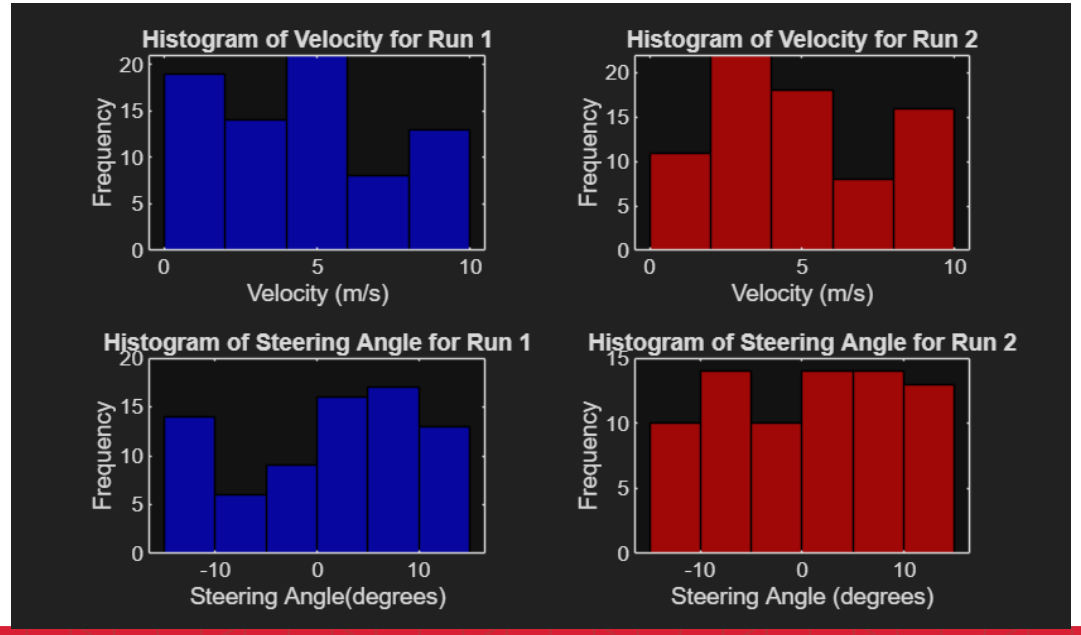
```
% Display smooth signal results
if var(avgVelocityRun1) < var(velocityRun1)
fprintf("\nRun 1 smoothing validated (variance reduced)\n");
else
fprintf("\nRun 1 smoothing may be incorrect\n");
end

if var(avgVelocityRun2) < var(velocityRun2)
fprintf("Run 2 smoothing validated (variance reduced)\n");
else
fprintf("Run 2 smoothing may be incorrect\n");
end
%stop events (velocity low)
stopEventsRun1 = sum(avgVelocityRun1 < stop);
stopEventsRun2 = sum(avgVelocityRun2 < stop);

% turning events (magnitude high)
turnEventsRun1 = sum(abs(avgSteeringAngleRun1 > turn));
turnEventsRun2 = sum(abs(avgSteeringAngleRun2 > turn));

%values (average speed)
avgSpeed1 = mean(avgVelocityRun1);
avgSpeed2 = mean(avgVelocityRun2);
```

```
% find sudden changes (spikes)
spikeThreshold = 7;
spikesRun1 = sum(abs(diff(avgSteeringAngleRun1)) > spikeThreshold);
spikesRun2 = sum(abs(diff(avgSteeringAngleRun2)) > spikeThreshold);

% Driving interpretation
if DQ1 > 80
    fprintf("Run 1 Behavior: Excellent control\n");
elseif DQ1 > 60
    fprintf("Run 1 Behavior: Moderate control\n");
elseif DQ1 > 40
    fprintf("Run 1 Behavior: Rough driving\n");
else
    fprintf("Run 1 Behavior: Poor control\n");
end

if DQ2 > 80
    fprintf("Run 2 Behavior: Excellent control\n");
elseif DQ2 > 60
    fprintf("Run 2 Behavior: Moderate control\n");
elseif DQ2 > 40
    fprintf("Run 2 Behavior: Rough driving\n");
else
    fprintf("Run 2 Behavior: Poor control\n");
end
```

Simulation Results (Plots)

- Include **at least 1-2 plots**
- Properly labeled:
  - Title
  - Axes
  - Units
- Examples:
  - Temperatures vs Time
  - Velocity vs. Time
  - Concentration vs. Time





Analysis of Results

- The graphs show velocity over time, steering over time, and the distribution of both metrics for each run.
- Any spikes or abrupt changes in these metrics were detected and documented.
- The models make sense given the randomized input velocity and steering angles.

Parameter Sensitivity / Exploration

- If you increased the time, the graphs would display a larger set of values. Changing thresholds for detection can increase or decrease sensitivity.
- Increasing sensitivity will increase the amount of total abrupt changes detected but will cause the detections to be less significant.
- Decreasing sensitivity will lead to only the largest changes being detected but could lead to overlooking smaller but still significant changes in velocity and steering.

Model Limitations

- The model uses randomized values for velocity and steering angle

  - Leads to more volatile changes in metrics that are inconsistent with typical driving habits

  - Plots seem chaotic compared to what would be expected from an authentic dataset.

Engineering Relevance

- This model has use in real world applications, being able to analyze someone's driving habits and determine safety levels.

- This model could also be used for testing robotics systems, finding the limits of a robot by analyzing when and where the system failed. (i.e. the robot rolling over when exceeding a specific steering angle beyond a velocity value).
  - The model finds locations of abrupt changes, so a large amount of data could be uploaded and sifted through autonomously.

Team Contributions

- Gracy Sylvestre:
  - Created the systems for determining changes in velocity and steering
  - Allowed for user input in time values and thresholds
  - Created the driving quality metric system
- Owen Rogg:
  - Created the input for velocity and steering
  - Created visuals for data
  - Set up dashboard for selecting and viewing specific plots

Conclusion

- Our team made a system able to analyze velocity and steering data, finding abrupt changes and assigning a driving quality score based on the data given.

- An integrated dashboard allows for the user to view each plot individually, and to input desired time values and thresholds.